LAB 6

PostgreSQL: Designing a Database Schema

Goal

Until now, the catalog's data has lived in the program's memory: the courses sat in a list in data.py (Lab 4), the enrollments in the ENROLLMENTS list in main.py (Lab 5), and all of it vanished on every backend restart. Today you move the catalog into PostgreSQL: you'll design a schema of three tables — `courses`, `students`, and `enrollments` — create it with SQL commands, load the data, and write the queries for every API route.

You'll work with the database directly, through the psql client and .sql files. The FastAPI and Next.js code doesn't change today: we'll connect the database to the backend in week 7 (SQLAlchemy). The second half of the lab is experiments. You'll deliberately violate every constraint and use the SQLSTATE code to decide which HTTP response the API would return, speed up a query with an index, and see a transaction cancel two inserts at once. The result is the course-catalog-db repository: schema.sql, seed.sql, queries.sql, and a README with an ER diagram and a log of your experiments.

Before you start

- the Lecture 6 handout, open nearby (sections 2–10: psql, types and keys, constraints, queries, indexes, transactions): today you'll repeat its examples almost word for word;
- PostgreSQL 16 or newer and the psql client (the lecture used version 18); if you don't have them, installation is described in section 1;
- a text editor for .sql files (VS Code works); save all files as UTF-8;
- git and a GitHub account: the result is submitted as a repository;
- the course-catalog and course-catalog-api projects don't change today;
- section 7 needs a second terminal window.

If you're opening psql for the first time

psql is PostgreSQL's console client: you type commands, it sends them to the server and prints the reply. There are two kinds of commands. SQL commands (`SELECT`, `INSERT`, `CREATE TABLE`) end with a semicolon. Meta-commands are handled by psql itself: they start with a backslash and need no semicolon. The minimum you'll need:

- the prompt shows where you are. In `course_catalog=>` you're connected to the course_catalog database and psql is waiting for a command. If there's a `#` instead of `>`, you're a superuser; if there's a `-` instead of `=`, the command isn't finished (usually a forgotten `;`). An asterisk before `>` (`=*>`) means “a transaction is open”, an exclamation mark (`=!>`) means “the transaction was aborted by an error” (section 7);
- meta-commands: `\c database role` — reconnect to another database or as another role, `\dt` — list tables, `\d table` — describe a table, `\i file.sql` — run a file, `\q` — quit;
- `\i` looks for the file relative to the folder you started psql from, so start psql from the project folder. On Windows, write paths with ordinary forward slashes;
- an error doesn't close psql. Read the message from the first word: after ERROR comes the cause, DETAIL shows which value didn't fit, and when you run a file, the file name and line number come before ERROR;
- if the prompt changed to `->` and your command is already complete, you forgot the `;`: add it, or press Ctrl+C to cancel the input;

- the up arrow brings back commands you typed earlier.

What you'll build

A repository with four files and three databases on the server: the main one, course_catalog, and two throwaway ones, idx_demo and norm_demo, which you can break and recreate without risking the main one. The work consists of eight steps; the table links them to files and lecture sections:

Step	What you do	What you get	Lecture
1	install and configure PostgreSQL, create a role and the databases	role `catalog_app`, database `course_catalog`	section 2
2	describe the tables, keys, and constraints	`schema.sql`	sections 3–6
3	load the initial data	`seed.sql`	section 3
4	write the queries for the API routes	`queries.sql`	section 7
5	violate every constraint and analyze the errors	a violation log in the README	sections 5–6
6	find a query that needs an index and look at EXPLAIN	an index in `schema.sql`, plans in the README	section 9
7	run two inserts in one transaction and roll them back	an entry in the README	section 10
8	assemble the result and review the schema against the checklist	a README with an ER diagram, a repository on GitHub	section 12

The repository structure by the end of the lab:

```
repository structure
course-catalog-db/
├── schema.sql              # tables, keys, constraints, the index
├── seed.sql                # initial data
├── queries.sql             # queries for the API routes
├── experiments/
│   └── idx_data.sql        # many rows for the index experiment (step 6)
└── README.md               # how to run, ER diagram, experiment logs
```

1. PostgreSQL, a Role, and a Database

First you need a running PostgreSQL server and the psql client. Check whether you have them:

```
terminal
psql --version
```

```
expected output (your version may differ)
psql (PostgreSQL) 18.4
```

You need version 16 or newer. If the command isn't found, install PostgreSQL in one of the ways below (all versions and systems are listed on the postgresql.org/download page). The second column shows how to open psql as the `postgres` superuser: through it you'll create the role and the databases.

System	How to install	How to open psql as the superuser
Windows	the installer from `postgresql.org/download/windows` (built by EDB); leave the port at 5432, and make up a password for the `postgres` user and remember it	Start menu → SQL Shell (psql): press Enter at the Server, Database, Port, and Username prompts, then enter the password. Or add the folder `C:\Program Files\PostgreSQL\18\bin` (the number is your version) to the PATH variable and run `psql -U postgres`
macOS	Homebrew: `brew install postgresql@18`, then `brew services start postgresql@18` (brew will tell you which line to add to PATH); the Postgres.app application works too	`psql postgres` (the superuser is your macOS user; no password needed)
Linux (Debian, Ubuntu)	`sudo apt install postgresql`; if it installs a version below 16, take the packages from `postgresql.org/download/linux`	`sudo -u postgres psql`
Docker	if Docker is already installed: `docker run --name pg -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres:18`	`psql -h localhost -U postgres`; the psql client must be installed on your computer, because the .sql files are on your machine, not in the container

> **Windows and non-English letters:** all the files in this lab are in English, so the console encoding won't cause trouble on its own. It matters when you add text with other letters, for example a name in Cyrillic: the Windows console doesn't use UTF-8 by default (psql may print a warning at startup: “Console code page … differs from Windows code page …”), and such text ends up in the database garbled, without any error. To avoid that, before starting psql run `$env:PGCLIENTENCODING = "UTF8"` in PowerShell (in cmd: `set PGCLIENTENCODING=UTF8`), or, in SQL Shell, where you can't set the variable, the command `\encoding UTF8` inside psql. If letters in the console still look strange, that doesn't mean they're corrupted in the database (in cmd, the command `chcp 65001` helps).

The role and the databases

The application doesn't need superuser rights (lecture section 2), so we'll create a role, `catalog_app`, that owns only our databases. While you're at it, create two throwaway databases for experiments: `idx_demo` is needed in section 6, `norm_demo` in a bonus task. Be sure to specify `OWNER catalog_app`: without it, the working role can't create tables (see “Common Mistakes”). Run everything in psql opened as the superuser:

```
psql — superuser
postgres=# SHOW server_version;
 server_version
----------------
 18.4
(1 row)

postgres=# CREATE ROLE catalog_app LOGIN PASSWORD 'change-me';
CREATE ROLE
postgres=# CREATE DATABASE course_catalog OWNER catalog_app;
CREATE DATABASE
postgres=# CREATE DATABASE idx_demo OWNER catalog_app;
CREATE DATABASE
postgres=# CREATE DATABASE norm_demo OWNER catalog_app;
CREATE DATABASE
postgres=# \c course_catalog catalog_app
You are now connected to database "course_catalog" as user "catalog_app".
course_catalog=> SELECT current_database(), current_user;
 current_database | current_user
------------------+--------------
 course_catalog   | catalog_app
(1 row)
```

```
course_catalog=> \dt
Did not find any tables.
```

`SHOW server_version` confirms the server version (yours may differ, but not below 16). The password is for teaching only; real passwords must never end up in git. If psql asks for the password for catalog_app after `\c`, enter `change-me` (the characters aren't shown as you type). `\dt` reports that there are no tables yet: psql versions before 18 print “Did not find any relations” here, and “List of relations” in table headings.

> **Check yourself:** `SELECT` returned course_catalog and catalog_app, and the prompt became `course_catalog=>` (a `>` instead of `#`: you're no longer a superuser). If not, repeat `\c course_catalog catalog_app`.

From here on you work as catalog_app. Quit psql with `\q`. From the next section on, you start psql from the project folder (it will appear in section 2) with this command; the password is `change-me`:

```
terminal — from the project folder
psql -h localhost -U catalog_app -d course_catalog
```

2. The Repository and schema.sql

Create the project folder, a git repository, and a folder for experiments:

```
terminal
mkdir course-catalog-db
cd course-catalog-db
git init
mkdir experiments
```

Open the folder in your editor and create the file schema.sql. It holds the three tables from lecture section 3. You choose the type, key, and constraints for each column yourself, using lecture sections 4–6; the domain rules are in the table:

Table	Column	Rule
courses	id	the course slug (for example, `modern-frontend`): text, primary key
courses	title, description	text, required
courses	credits	an integer, required, strictly greater than 0
courses	is_elective	yes/no, required, defaults to “no”
courses	likes	an integer, required, defaults to 0, not less than 0
students	id	an automatic number, 8 bytes (`bigint`), primary key
students	name	text, required, at least 2 characters long
students	email	text, required, no two students share one
enrollments	id	an automatic number (`bigint`), primary key
enrollments	course_id	required, references `courses`; a course that has enrollments can't be deleted

Table	Column	Rule
enrollments	student_id	required, references `students`; when a student is deleted, their enrollments are deleted
enrollments	enrolled_at	the moment of enrollment (with time zone), required, defaults to the current moment
enrollments	(course_id, student_id)	the pair isn't repeated: a student enrolls in a course only once

How to write a rule in SQL — a cheat sheet (details in lecture section 6):

Rule	How to write it
the value is required	`NOT NULL`
a default value	`DEFAULT false`, `DEFAULT 0`, `DEFAULT now()`
the value must satisfy a condition	`CHECK (credits > 0)`
the value isn't repeated (including a pair of columns)	`UNIQUE`, `UNIQUE (course_id, student_id)`
an automatic number and a primary key	`bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY`
a reference to another table and the behavior on deletion	`REFERENCES courses (id) ON DELETE RESTRICT` (or `CASCADE`)

The skeleton for schema.sql. Each TODO is one rule from the table above; replace the column and the comment above it with the finished definition:

```
schema.sql — skeleton
-- schema.sql: the course catalog database schema
-- The file can be run repeatedly: it first drops the old tables.
-- A teaching shortcut: real projects manage the schema with migrations.
DROP TABLE IF EXISTS enrollments, students, courses;

CREATE TABLE courses (
    id          text    PRIMARY KEY,
    title       text    NOT NULL,
    description text    NOT NULL,
    -- TODO 1: credits — required; strictly greater than zero
    credits     integer,
    -- TODO 2: is_elective — required; defaults to false
    is_elective boolean,
    -- TODO 3: likes — required; defaults to 0; not less than zero
    likes       integer
);

CREATE TABLE students (
    -- TODO 4: id — an automatic number (bigint) and the primary key
    id    bigint,
    -- TODO 5: name — required; at least two characters long
    name  text,
    -- TODO 6: email — required; unique
    email text
);
```

```
CREATE TABLE enrollments (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    -- TODO 7: course_id — required; references courses;
    --         a course that has enrollments can't be deleted
    course_id   text,
    -- TODO 8: student_id — required; references students;
    --         when a student is deleted, their enrollments are deleted
    student_id  bigint,
    -- TODO 9: enrolled_at — required; defaults to the current moment
    enrolled_at timestamptz,
    -- TODO 10: the pair (course_id, student_id) is unique
    --          (enrolled_at must end with a comma)
);
```

> **The order of the tables matters:** a foreign key can only reference a table that already exists, so `courses` and `students` are created before `enrollments`. You'll need the same order in seed.sql, and the first command in the file, `DROP TABLE`, drops all three tables at once.

Run the file. Start psql from the project folder and enter `\i schema.sql`:

```
terminal — from the project folder
psql -h localhost -U catalog_app -d course_catalog
```

```
psql
course_catalog=> \i schema.sql
psql:schema.sql:4: NOTICE:  table "enrollments" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "students" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "courses" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
course_catalog=> \dt
              List of tables
 Schema |    Name     | Type  |    Owner
--------+-------------+-------+-------------
 public | courses     | table | catalog_app
 public | enrollments | table | catalog_app
 public | students    | table | catalog_app
(3 rows)
```

The first run prints three NOTICE lines: “the table doesn't exist, skipping”. These aren't errors: `DROP TABLE IF EXISTS` drops the old tables if there are any, so the file can be run as many times as you like, and you'll keep fixing the schema as you go. Real projects don't do this, because the data would vanish along with the tables;

there, migrations manage the schema (week 7). Now check what you got: `\d enrollments` shows the columns, indexes, and keys of the most complex table. Long lines here and later are wrapped for print.

```
psql
course_catalog=> \d enrollments
                                  Table "public.enrollments"
   Column    |           Type           | Collation | Nullable |           Default
-------------+--------------------------+-----------+----------+------------------------------
 id          | bigint                   |           | not null | generated always as identity
 course_id   | text                     |           | not null |
 student_id  | bigint                   |           | not null |
 enrolled_at | timestamp with time zone |           | not null | now()
Indexes:
    "enrollments_pkey" PRIMARY KEY, btree (id)
    "enrollments_course_id_student_id_key" UNIQUE CONSTRAINT, btree (course_id, student_id)
Foreign-key constraints:
    "enrollments_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE
        RESTRICT
    "enrollments_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE
        CASCADE
```

For `courses` and `students`, run `\d courses` and `\d students` and compare them with the rules table. `courses` should have two CHECK constraints (`courses_credits_check` and `courses_likes_check`), and `students` a UNIQUE constraint on email (`students_email_key`) and a CHECK on the name (`students_name_check`). PostgreSQL built these names itself; you'll need them in section 5.

> **Check yourself:** `\dt` shows three tables; in `\d enrollments` all columns are not null, with two indexes and two foreign keys — `ON DELETE RESTRICT` to courses and `ON DELETE CASCADE` to students. If not, fix schema.sql and run it again: it drops the old tables itself.

> **An error in the middle of the file:** by default psql doesn't stop at an error and runs the remaining commands. Find the very first ERROR in the output (it names the file line), fix it, and run the file again. To make execution stop at the first error, run the file like this: `psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f schema.sql`.

When the schema looks right, make your first commit:

```
terminal
git add schema.sql
git commit -m "Add schema.sql with constraints"
```

3. seed.sql: the Initial Data

The tables are empty. Fill them with data from earlier labs: six courses from data.py (Labs 4 and 5), three students, and six enrollments. The courses are already written in the skeleton; you write the students and enrollments yourself from the table below.

Student	Email	Enrolled in courses
Aigerim Sadykova	aigerim@example.com	modern-frontend, ai-integration
Daniyar Bekov	daniyar@example.com	modern-frontend, backend-fastapi, ai-integration
Asel Omarova	asel@example.com	ai-integration

```
seed.sql — skeleton
-- seed.sql: initial data (the courses come from data.py in Labs 4 and 5)

INSERT INTO courses (id, title, description, credits, is_elective, likes) VALUES
  ('modern-frontend', 'Modern Frontend: React & Next.js',
   'React 19, Server Components, and the App Router.', 5, false, 24),
  ('backend-fastapi', 'Backend Foundations: FastAPI',
   'Async REST APIs in Python with FastAPI and Pydantic.', 5, false, 19),
  ('databases-postgresql', 'Relational Databases: PostgreSQL',
   'Schemas, SQLAlchemy, and migrations with Alembic.', 5, false, 15),
  ('api-design', 'API Design: REST vs GraphQL',
   'Comparing REST and GraphQL in one hands-on project.', 4, true, 11),
  ('web-security', 'Web Security Essentials',
   'JWT/OAuth2, XSS, CSRF, and SQL injection defense.', 4, false, 21),
  ('ai-integration', 'AI/LLM Integration',
   'LLM features in an app via the OpenAI API.', 5, true, 32);

-- TODO 1: add three students (name and email from the table).
--         Don't specify the id column: the database issues the number
INSERT INTO students (name, email) VALUES
  ...

-- TODO 2: add six enrollments: course_id and the student's number
--         (the counter issued 1, 2, 3 in insertion order)
INSERT INTO enrollments (course_id, student_id) VALUES
  ...
```

> **The insertion order:** first `courses` and `students`, then `enrollments`: an enrollment references rows that must already exist. The students get the numbers 1, 2, and 3 from the counter because the table has just been created; if you ran seed.sql twice, run schema.sql again — it recreates the tables, and the counter starts from one.

Run the file. In the reply `INSERT 0 6`, the second number matters — how many rows were inserted (the first is always 0):

```
psql
course_catalog=> \i seed.sql
INSERT 0 6
INSERT 0 3
INSERT 0 6
course_catalog=> SELECT (SELECT count(*) FROM courses) AS courses,
course_catalog->        (SELECT count(*) FROM students) AS students,
course_catalog->        (SELECT count(*) FROM enrollments) AS enrollments;
 courses | students | enrollments
---------+----------+-------------
       6 |        3 |           6
(1 row)
```

Check the contents too: print the courses and the students with queries.

```
psql
course_catalog=> SELECT id, credits, is_elective, likes FROM courses ORDER BY id;
          id          | credits | is_elective | likes
----------------------+---------+-------------+-------
 ai-integration       |       5 | t           |    32
 api-design           |       4 | t           |    11
 backend-fastapi      |       5 | f           |    19
 databases-postgresql |       5 | f           |    15
 modern-frontend      |       5 | f           |    24
 web-security         |       4 | f           |    21
(6 rows)

course_catalog=> SELECT id, name, email FROM students ORDER BY id;
 id |       name       |        email
----+------------------+---------------------
  1 | Aigerim Sadykova | aigerim@example.com
  2 | Daniyar Bekov    | daniyar@example.com
  3 | Asel Omarova     | asel@example.com
(3 rows)
```

> **Check yourself:** the counters show 6, 3, and 6; there are two elective courses (`ai-integration` and `api-design`); the students have numbers 1, 2, and 3. If the row counts differ, run schema.sql and seed.sql again: that starts you from a clean state.

Make a commit:

```
terminal
git add seed.sql
git commit -m "Add seed data"
```

4. queries.sql: Queries for the API Routes

In Lab 5, each FastAPI route worked with a Python list in memory. Now each route needs a database query. You'll write the queries into the file queries.sql and test them in psql. We'll connect the database to FastAPI in week 7; first make sure the SQL works on its own: errors are easier to find where there's no application code.

Query	Route	What it does
1, 1b	`GET /courses`	a page of the course list: filter by `is_elective`, sort by popularity
2, 2b	`GET /courses/{course_id}`	one course; for a nonexistent one, an empty result
3, 3b	`POST /courses/{course_id}/like`	a like: the counter goes up by one
4	`POST /enrollments`	enroll a student in a course
5	`GET /enrollments`	all enrollments with the student's name and the course title
6	there's no route for it yet	statistics: how many enrollments each course has, including courses with none

> **The values are written right into the query text:** `'api-design'`, `3`, and so on — it's easier to test a query in psql that way. In application code, values are passed separately from the query text; otherwise SQL injection is possible (bonus task 3 and week 7).
>
> **An enrollment now references the student by number.** In Lab 5, an enrollment stored a name (`student_name`); now it holds the student's number (`student_id`) from the `students` table. We'll rewrite the API in week 7, so query 4 takes the student's number.
>
> **Queries 3 and 4 are wrapped in `BEGIN … ROLLBACK`.** They change data, and this way the file can be run again and again while the data stays the same. You'll learn what these commands do in section 7; for now, leave them as in the skeleton.

The course list: GET /courses

Query 1 repeats `list_courses` from Lab 5: a filter on `is_elective`, sorting by popularity, and a page of three courses. The `pagination` function computed the offset as `(page - 1) * page_size`; in SQL, that corresponds to `OFFSET`. Query 1b is the second page of the same data. Create the file queries.sql:

```
queries.sql — skeleton, part 1
-- queries.sql: queries for the API routes (values are filled in as examples)
-- Queries that change data are wrapped in a transaction that rolls back,
-- so the file can be run as many times as you like.

-- 1. GET /courses?is_elective=false&sort=popular&page=1&page_size=3
-- TODO 1: id, title, credits, likes; non-elective courses only;
--         most popular first (ties on likes — by id);
--         LIMIT and OFFSET for page 1

-- 1b. the same, page 2: OFFSET = (2 - 1) * 3
-- TODO 2: the same query for page 2
```

Hints:

- the order of a query's parts: `SELECT … FROM … WHERE … ORDER BY … LIMIT … OFFSET`; the sort direction `DESC` goes after the column;
- a second sort column (`id`) makes the order unambiguous: if two courses have the same number of likes, PostgreSQL is free to return them in any order, and with pagination a course can land on both pages or on neither;
- for `sort=title`, the sort would be `ORDER BY title`; you don't need to add that variant to the file.

Run the queries in psql (copy them from the file, or enter `\i queries.sql` once you've written them all). Expected result:

```
expected result of queries 1 and 1b
       id        |              title               | credits | likes
-----------------+----------------------------------+---------+-------
 modern-frontend | Modern Frontend: React & Next.js |       5 |    24
 web-security    | Web Security Essentials          |       4 |    21
 backend-fastapi | Backend Foundations: FastAPI     |       5 |    19
(3 rows)

         id          |              title               | credits | likes
---------------------+----------------------------------+---------+-------
 databases-postgresql | Relational Databases: PostgreSQL |       5 |    15
(1 row)
```

Check yourself: the first page has three courses with 24, 21, and 19 likes, and none of them is an elective; the second page has one course left — `databases-postgresql` (15 likes). If the order differs, check `DESC` and `WHERE`.

One course and a like: GET /courses/{course_id}, POST …/like

Query 2 corresponds to the `get_course` route, query 3 to `like_course`. Each needs a variant with a nonexistent course: that's how you'll see how the database says “not found”. Add to queries.sql:

```
queries.sql — skeleton, part 2
-- 2. GET /courses/{course_id}
-- TODO 3: id, title, credits, is_elective, likes of the course 'api-design'

-- 2b. no such course: 0 rows, the API answers 404
-- TODO 4: the same query for 'no-such-course'

-- 3. POST /courses/{course_id}/like
BEGIN;
-- TODO 5: increase likes by 1 for 'api-design' with a single UPDATE
--         and return the id and the new value (RETURNING)
-- 3b. no such course: UPDATE 0, the API answers 404
-- TODO 6: the same for 'no-such-course'
ROLLBACK;
```

Hints:

- strings in SQL are written in single quotes: `WHERE id = 'api-design'`;
- `UPDATE … SET likes = likes + 1 WHERE … RETURNING id, likes` increases the counter and immediately returns the new value; without `WHERE`, every course would get the like at once;
- `BEGIN;` and `ROLLBACK;` are already written; you write the two `UPDATE` commands between them.

Single and double quotes are not the same thing. Strings go in single quotes. In PostgreSQL, double quotes mean a column or table name: the query `WHERE id = "api-design"` fails with `column "api-design" does not exist`, because the database looks for a column with that name.

```
expected result of queries 2, 2b, and 3
     id     |           title            | credits | is_elective | likes
------------+----------------------------+---------+-------------+-------
 api-design | API Design: REST vs GraphQL |       4 | t           |    11
(1 row)

 id | title | credits | is_elective | likes
----+-------+---------+-------------+-------
(0 rows)

BEGIN
     id     | likes
------------+-------
 api-design |    12
(1 row)

UPDATE 1
 id | likes
----+-------
(0 rows)

UPDATE 0
ROLLBACK
```

Check yourself: query 2 returned one row (`api-design` has 4 credits, and the letter `t` in the `is_elective` column means true), query 2b returned `(0 rows)`; in query 3 the likes became 12, and for the nonexistent course `UPDATE 0` came back. After `ROLLBACK`, `api-design` has 11 likes again — verify by repeating query 2.

`(0 rows)` and `UPDATE 0` are not errors. For a database, the absence of a row is an ordinary result. What to answer the client is up to the application: if there are no rows, it returns 404. The like increases the counter inside the command itself (`likes = likes + 1`) rather than reading the value into the program and writing it back; why that matters, you'll see in bonus task 2.

Enrolling and listing enrollments: POST and GET /enrollments

Query 4 creates an enrollment; query 5 repeats `list_enrollments`, but now with real student names and course titles — for that, the tables are joined with `JOIN`. Add to queries.sql:

```
queries.sql — skeleton, part 3
-- 4. POST /enrollments
BEGIN;
-- TODO 7: enroll the student with number 3 in the course 'web-security';
--         return id, course_id, and student_id (RETURNING)
ROLLBACK;

-- 5. GET /enrollments
-- TODO 8: the enrollment number (id), the student's name (AS student), and
--         the course title (AS course); join the tables with JOIN;
--         order by the enrollment number
```

Hints:

- in the `INSERT`, we don't specify the `id` and `enrolled_at` columns: the database fills in the number and the current moment;
- query 5 needs two `JOIN`s: `JOIN students s ON s.id = e.student_id` and the same for `courses`; short aliases (`e`, `s`, `c`) keep the query readable, and `AS student` and `AS course` set the names of the result columns.

```
expected result of queries 4 and 5
BEGIN
 id |  course_id   | student_id
----+--------------+------------
  7 | web-security |          3
(1 row)

INSERT 0 1
ROLLBACK
 id |     student      |              course
----+------------------+----------------------------------
  1 | Aigerim Sadykova | Modern Frontend: React & Next.js
  2 | Aigerim Sadykova | AI/LLM Integration
  3 | Daniyar Bekov    | Modern Frontend: React & Next.js
  4 | Daniyar Bekov    | Backend Foundations: FastAPI
  5 | Daniyar Bekov    | AI/LLM Integration
  6 | Asel Omarova     | AI/LLM Integration
(6 rows)
```

Your enrollment number may differ. A rollback undoes the insert but not the counter: PostgreSQL issues numbers outside of transactions, so each run of query 4 uses up the next number (7, then 8, and so on). The numbers stay unique but needn't be consecutive.

Course statistics

The last query is new: how many enrollments each course has. Courses with no enrollments must also appear in the result, with a zero. Add to queries.sql:

```
queries.sql — skeleton, part 4
-- 6. statistics: enrollments per course, including courses with none
-- TODO 9: LEFT JOIN enrollments to courses; GROUP BY the course;
--         the number of enrollments — count(e.id) AS enrolled;
--         courses with the most enrollments first (ties — by id)
```

```
expected result of query 6
          id          | enrolled
----------------------+----------
 ai-integration       |        3
 modern-frontend      |        2
 backend-fastapi      |        1
 api-design           |        0
 databases-postgresql |        0
 web-security         |        0
(6 rows)
```

It's easy to get this query wrong by a single function. For comparison, run the `count(*)` variant in psql (you don't need to add it to the file):

```
psql
course_catalog=> SELECT c.id, count(*) AS wrong, count(e.id) AS enrolled
course_catalog-> FROM courses c
course_catalog-> LEFT JOIN enrollments e ON e.course_id = c.id
course_catalog-> GROUP BY c.id
course_catalog-> ORDER BY enrolled DESC, c.id;
          id          | wrong | enrolled
----------------------+-------+----------
 ai-integration       |     3 |        3
 modern-frontend      |     2 |        2
 backend-fastapi      |     1 |        1
 api-design           |     1 |        0
 databases-postgresql |     1 |        0
 web-security         |     1 |        0
(6 rows)
```

For courses with no enrollments, `LEFT JOIN` returns one row in which the enrollment's fields are empty (NULL). `count(*)` counts rows, so it counted one in the `wrong` column; `count(e.id)` counts only non-empty values and correctly shows 0. The rule: with `LEFT JOIN`, count a column of the right-hand table.

When all nine queries are written, run the whole file: start psql from the project folder and enter `\i queries.sql`. The results should match those shown above (except for the enrollment number in query 4), and none of them should be ERROR messages. Then make a commit:

```
terminal
git add queries.sql
git commit -m "Add queries for API routes"
```

5. Violating the Constraints and Reading the Errors

Constraints protect the data only if the application can tell which one fired. In this step you violate each of the schema's rules in turn and record three things: the five-character SQLSTATE code, the name of the constraint that fired, and the HTTP response the API should return. The table mapping codes to responses is in lecture section 6. You'll keep the records in README.md, in the “Constraint violation log” section.

Verbose error output

By default, psql shows the error text without the code and without the constraint name. Turn on verbose mode (it lasts until the end of the psql session) and run the first command from the list — inserting a course without a title:

```
psql
course_catalog=> \set VERBOSITY verbose
course_catalog=> INSERT INTO courses (id, title, description, credits)
course_catalog-> VALUES ('x-test', NULL, 'Test', 3);
ERROR:  23502: null value in column "title" of relation "courses" violates not-null constraint
DETAIL:  Failing row contains (x-test, null, Test, 3, f, 0).
SCHEMA NAME:  public
TABLE NAME:  courses
COLUMN NAME:  title
LOCATION:  ReportNotNullViolationError, execMain.c:2219
```

Let's go through the reply line by line:

- `ERROR:  23502:` — the word ERROR and the SQLSTATE code. The first two digits (23) are the class “integrity constraint violation”: all the schema's constraints produce codes that start this way;
- the text after the code says what happened: a null value (`null value`) in the `title` column of the `courses` table violates NOT NULL;
- `DETAIL` shows the whole row that was being inserted: the values are in table column order. The columns we didn't specify got default values: we set `credits` = 3 ourselves, while `is_elective` = `f` (false) and `likes` = 0 were filled in by the database;
- `SCHEMA NAME`, `TABLE NAME`, `COLUMN NAME`, and `CONSTRAINT NAME` are the same information as separate fields: a program uses them to identify the constraint that fired without parsing the message text. NOT NULL has no name of its own, so the column is shown (`COLUMN NAME`);
- `LOCATION` is a place in the server's source code. It means nothing to you, and the line numbers differ between PostgreSQL versions.

> **Forgot to turn on verbose mode?** Right after the error, run `\errverbose`: psql repeats the last error with all its fields. Turning on `\set VERBOSITY verbose` beforehand is more convenient: you see the code in every error.

How to run an experiment

Break one thing at a time and act the same way each time:

1. Run a command from the list below. It should end with an error: the inserts and updates here are deliberately wrong.
2. Find the code after `ERROR:` and the name in `CONSTRAINT NAME` in the reply (for NOT NULL, the column from `COLUMN NAME`).
3. Decide which HTTP response the API would return, and write everything down in the log.

4. Move on to the next command. After an error outside a transaction, psql is immediately ready for a new command, and the data hasn't changed.

> **The command ran without an error?** Then your schema lacks the rule, and the command managed to change the data. Fix schema.sql, run `\i schema.sql` and `\i seed.sql` (they recreate the tables with clean data), and repeat the command.

```
commands for the log — enter them one at a time
-- 1. NULL in a required column
INSERT INTO courses (id, title, description, credits)
VALUES ('x-test', NULL, 'Test', 3);
-- 2. zero credits
INSERT INTO courses (id, title, description, credits)
VALUES ('x-test', 'Test', 'Test', 0);
-- 3. a negative number of likes
UPDATE courses SET likes = -1 WHERE id = 'api-design';
-- 4. a too-short student name
INSERT INTO students (name, email) VALUES ('A', 'a@example.com');
-- 5. a duplicate email
INSERT INTO students (name, email)
VALUES ('Dup Student', 'aigerim@example.com');
-- 6. a duplicate course id
INSERT INTO courses (id, title, description, credits)
VALUES ('api-design', 'x', 'y', 1);
-- 7. enrolling a student in the same course again
INSERT INTO enrollments (course_id, student_id)
VALUES ('modern-frontend', 1);
-- 8. enrolling in a nonexistent course
INSERT INTO enrollments (course_id, student_id)
VALUES ('no-such-course', 1);
-- 9. enrolling a nonexistent student
INSERT INTO enrollments (course_id, student_id)
VALUES ('web-security', 999);
-- 10. deleting a course that has enrollments
DELETE FROM courses WHERE id = 'modern-frontend';
```

For comparison, here are two more replies: to the duplicate email (command 5) and to deleting a course with enrollments (command 10). You'll get the rest yourself.

```
psql
course_catalog=> INSERT INTO students (name, email)
course_catalog-> VALUES ('Dup Student', 'aigerim@example.com');
ERROR:  23505: duplicate key value violates unique constraint "students_email_key"
DETAIL:  Key (email)=(aigerim@example.com) already exists.
SCHEMA NAME:  public
TABLE NAME:  students
CONSTRAINT NAME:  students_email_key
LOCATION:  _bt_check_unique, nbtinsert.c:673
course_catalog=> DELETE FROM courses WHERE id = 'modern-frontend';
ERROR:  23001: update or delete on table "courses" violates RESTRICT setting of foreign key
        constraint "enrollments_course_id_fkey" on table "enrollments"
DETAIL:  Key (id)=(modern-frontend) is referenced from table "enrollments".
SCHEMA NAME:  public
TABLE NAME:  enrollments
CONSTRAINT NAME:  enrollments_course_id_fkey
LOCATION:  ri_ReportViolation, ri_triggers.c:2797
```

Keep the log in the project's README.md using this template. The first two columns are filled in; the last three are your observations:

```
README.md — log template
## Constraint violation log

Commands were run with `\set VERBOSITY verbose`.

| # | What we violate | SQLSTATE | Constraint | API response |
|---|-----------------|----------|------------|--------------|
| 1 | NULL in courses.title | | | |
| 2 | courses.credits = 0 | | | |
| 3 | courses.likes = -1 | | | |
| 4 | a one-letter student name | | | |
| 5 | a duplicate student email | | | |
| 6 | a duplicate course id | | | |
| 7 | a repeated enrollment in a course | | | |
| 8 | enrolling in a nonexistent course | | | |
| 9 | enrolling a nonexistent student | | | |
| 10 | deleting a course that has enrollments | | | |
```

Cascading delete: the rule that doesn't break

One schema rule doesn't produce an error: `ON DELETE CASCADE` on `enrollments.student_id`. When a student is deleted, the database deletes their enrollments itself. Check it, and so that the data doesn't disappear, run the experiment inside a transaction and cancel it with `ROLLBACK`:

```
psql
course_catalog=> BEGIN;
BEGIN
course_catalog=*> SELECT count(*) AS enrollments FROM enrollments;
 enrollments
-------------
           6
(1 row)

course_catalog=*> DELETE FROM students WHERE email = 'daniyar@example.com';
DELETE 1
course_catalog=*> SELECT count(*) AS enrollments FROM enrollments;
 enrollments
-------------
           3
(1 row)

course_catalog=*> ROLLBACK;
ROLLBACK
course_catalog=> SELECT count(*) AS enrollments FROM enrollments;
 enrollments
-------------
           6
(1 row)
```

There were six enrollments; after deleting Daniyar, three were left: the student's three enrollments went with him. After `ROLLBACK`, there are six again. Add row 11 to the log with the outcome (“deleting a student: the enrollments were deleted by cascade, 6 → 3, 6 after ROLLBACK”).

Check your log

Fill in the log and only then compare it with this table. The API responses follow the table in lecture section 6.

#	SQLSTATE	Constraint	API response
1	`23502`	NOT NULL has no name of its own: the message names the column `title` (the `COLUMN NAME` field)	422; if Pydantic let the empty value through, that's a bug in the code (500)
2	`23514`	`courses_credits_check`	422
3	`23514`	`courses_likes_check`	500: only our code changes the counter (`likes + 1`), so it must never go negative — if it does, there's a bug in the code
4	`23514`	`students_name_check`	422
5	`23505`	`students_email_key`	409 Conflict
6	`23505`	`courses_pkey`	409 Conflict
7	`23505`	`enrollments_course_id_student_id_key`	409 Conflict
8	`23503`	`enrollments_course_id_fkey`	404: course not found
9	`23503`	`enrollments_student_id_fkey`	404: student not found (422 also works; write down your choice)
10	`23001` (PostgreSQL 18) or `23503` (16 and 17)	`enrollments_course_id_fkey`	409 Conflict: a course with enrollments can't be deleted

The server version affects the error text and code. On PostgreSQL 18, a delete blocked by `RESTRICT` gives the message “violates RESTRICT setting of foreign key constraint” and code 23001, while on versions 16 and 17 it's “violates foreign key constraint” and code 23503. That's why errors are told apart by the code class (they all start with 23) and the constraint name, not by the message text.

Gaps in the numbers are not an error. In the reply to command 4 you saw `Failing row contains (4, A, a@example.com)`: the number 4 was issued to the student before the constraints were checked, and it isn't returned to the counter after the error. Experiments 4 and 5 each used up a number, so the next student will get 6, not 4. A `GENERATED ALWAYS AS IDENTITY` column guarantees that numbers are unique, not that there are no gaps. Your numbers may differ.

Check yourself: the tables still have 6, 3, and 6 rows: none of the commands changed the data. Check with the query `SELECT count(*) FROM courses;` and the same for the other tables.

An extra check (optional): a manual number

The `students.id` column is declared `GENERATED ALWAYS`, so you can't set a number by hand. Try it, and note the code: it isn't from class 23.

```
psql
course_catalog=> INSERT INTO students (id, name, email)
course_catalog-> VALUES (100, 'Test Student', 'test@example.com');
ERROR:  428C9: cannot insert a non-DEFAULT value into column "id"
DETAIL:  Column "id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
LOCATION:  rewriteTargetListIU, rewriteHandler.c:948
course_catalog=> \set VERBOSITY default
```

The code `428C9` means an attempt to write a value into a column whose value the database always issues. For the API, that isn't a user error but a bug in the code: a 500 response. After the experiment, verbose mode is turned off with `\set VERBOSITY default`.

A question for the README

Pydantic in the API already checks the name length (`Field(min_length=2)`), and the database has the same rule written again (`students_name_check`). Add two or three sentences to the README: why check in the database if there's a check in code? The answer is in lecture section 6 (a hint: think about who else writes to the database besides your API).

Make a commit:

```
terminal
git add README.md
git commit -m "Add constraint violation log"
```

6. An Index: a Slow Query and EXPLAIN Before and After

The database reads a six-row table instantly, so an index is useless there. To see its benefit, you'll take a separate database, `idx_demo` (you created it in section 1), fill it with data, and compare the plan of the same query before and after creating the index. This experiment doesn't touch the main database, course_catalog.

Preparation: lots of data

Create the file experiments/idx_data.sql. It inserts 500 courses, 20,000 students, and 50,000 enrollments and refreshes the statistics. The numbers come from `generate_series(1, N)`: it returns the sequence 1, 2, …, N, and `INSERT … SELECT` inserts one row for each number (`||` concatenates strings). You don't need to invent formulas: copy the file as is.

```
experiments/idx_data.sql
-- idx_data.sql: lots of data for index experiments
-- (500 courses, 20,000 students, 50,000 enrollments)

INSERT INTO courses (id, title, description, credits)
SELECT 'course-' || g, 'Course ' || g, 'Test', 3
FROM generate_series(1, 500) AS g;

INSERT INTO students (name, email)
SELECT 'Student ' || g, 'student' || g || '@example.com'
FROM generate_series(1, 20000) AS g;

INSERT INTO enrollments (course_id, student_id)
SELECT 'course-' || (1 + (g + g / 20000) % 500), 1 + g % 20000
FROM generate_series(1, 50000) AS g;

-- after loading, refresh the statistics the planner relies on
ANALYZE courses;
ANALYZE students;
ANALYZE enrollments;
```

> **A schema without the index.** While you do this step, your schema.sql must not contain `CREATE INDEX`: the experiment is built on comparing "before" and "after". You'll add the index to the file after the experiment.

Start psql from the project folder, switch to the idx_demo database, and load the schema and the data:

```
psql
course_catalog=> \c idx_demo
You are now connected to database "idx_demo" as user "catalog_app".
idx_demo=> \i schema.sql
psql:schema.sql:4: NOTICE:  table "enrollments" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "students" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "courses" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
idx_demo=> \i experiments/idx_data.sql
INSERT 0 500
INSERT 0 20000
INSERT 0 50000
ANALYZE
ANALYZE
ANALYZE
idx_demo=> SELECT (SELECT count(*) FROM students) AS students,
idx_demo->        (SELECT count(*) FROM enrollments) AS enrollments;
 students | enrollments
----------+-------------
    20000 |       50000
(1 row)
```

`\c idx_demo` switches psql to another database under the same role. The three NOTICE lines are familiar: the new database has no tables yet. The `INSERT 0 500` lines and below are the numbers of inserted rows; `ANALYZE` means the database refreshed the statistics from which the planner estimates how many rows a query will return.

Three queries, three plans

`EXPLAIN` shows a query's plan — how the database intends to execute it — without running the query. In a plan, look at three things: the access method (`Seq Scan` — reading the whole table sequentially; `Index Scan` and `Bitmap … Scan` — reading through an index), the cost estimate `cost=from..to` (arbitrary units; the lower, the cheaper), and the condition (`Filter` or `Index Cond`). Run the three queries one at a time:

```
three queries — enter them one at a time
-- 1. look up a student by email
EXPLAIN SELECT * FROM students WHERE email = 'student777@example.com';
-- 2. enrollments in one course
EXPLAIN SELECT * FROM enrollments WHERE course_id = 'course-7';
-- 3. one student's enrollments
EXPLAIN SELECT * FROM enrollments WHERE student_id = 777;
```

Here's the reply to the first and third queries. Run the second one yourself and find out which index it uses.

```
psql
idx_demo=> EXPLAIN SELECT * FROM students WHERE email = 'student777@example.com';
                                  QUERY PLAN
-------------------------------------------------------------------------------------
 Index Scan using students_email_key on students  (cost=0.29..8.30 rows=1 width=45)
   Index Cond: (email = 'student777@example.com'::text)
(2 rows)

idx_demo=> EXPLAIN SELECT * FROM enrollments WHERE student_id = 777;
                         QUERY PLAN
------------------------------------------------------------
 Seq Scan on enrollments  (cost=0.00..1042.00 rows=3 width=34)
   Filter: (student_id = 777)
(2 rows)
```

Compare with the table (your cost and rows numbers may differ slightly — what matters is the access method):

Query	Plan	Where the index comes from
1. by `email`	`Index Scan using students_email_key`	a `UNIQUE` constraint creates an index by itself
2. by `course_id`	`Bitmap Index Scan on enrollments_course_id_student_id_key`	the index on the unique pair `(course_id, student_id)` also works for a search on just the pair's first column
3. by `student_id`	`Seq Scan on enrollments`, cost `0.00..1042.00`	`student_id` has no index: PostgreSQL creates indexes for `PRIMARY KEY` and `UNIQUE`, but not for `FOREIGN KEY`; and a search on the pair's second column isn't sped up

The weak spot is found: to locate a student's three enrollments, the database reads all 50,000 rows of the table.

Measuring the time: EXPLAIN ANALYZE

`EXPLAIN ANALYZE` not only builds the plan but also runs the query, adding the actual time. Run it for the `student_id` query three times in a row and note the smallest value from the `Execution Time` line (the first time, the data may not be in the cache yet):

```
psql
EXPLAIN ANALYZE SELECT * FROM enrollments WHERE student_id = 777;
```

The output is long; we're interested in two of its lines:

```
from the EXPLAIN ANALYZE output — before the index (the rest is omitted)
   Rows Removed by Filter: 49997
 Execution Time: 3.636 ms
```

`Rows Removed by Filter: 49997` — the database read and discarded almost the whole table for the sake of three needed rows. Your time will be different: it depends on your computer.

> **EXPLAIN ANALYZE really runs the query.** For `SELECT` that's safe, but `INSERT`, `UPDATE`, and `DELETE` will change the data. Test such queries inside `BEGIN … ROLLBACK`.

> **The output depends on the server version.** On PostgreSQL 18, `EXPLAIN ANALYZE` prints more lines (for example, `Buffers` and `Index Searches`) and a fractional row count (`rows=3.00`); versions 16 and 17 don't. That's a difference in the output, not in the plan.

Creating the index and comparing

An index is created with the command `CREATE INDEX name ON table (column)`; the convention is to build the name as `table_column_idx`. Create an index on `student_id` and look at the plan again:

```
psql
idx_demo=> CREATE INDEX enrollments_student_id_idx ON enrollments (student_id);
CREATE INDEX
idx_demo=> EXPLAIN SELECT * FROM enrollments WHERE student_id = 777;
                                      QUERY PLAN
---------------------------------------------------------------------------------------
 Bitmap Heap Scan on enrollments  (cost=4.31..15.59 rows=3 width=34)
   Recheck Cond: (student_id = 777)
   ->  Bitmap Index Scan on enrollments_student_id_idx  (cost=0.00..4.31 rows=3 width=0)
         Index Cond: (student_id = 777)
(4 rows)
```

The plan changed: instead of reading the whole table (`Seq Scan`, cost 1042), the database finds the needed rows through the index (`Bitmap Index Scan`, cost about 15). Repeat `EXPLAIN ANALYZE` three times and compare `Execution Time`: for us it dropped from a few milliseconds to hundredths of a millisecond — tens of times faster. Your numbers will be different, but the difference should be noticeable.

An index isn't free. See how much space it takes:

```
psql
idx_demo=> SELECT pg_size_pretty(pg_relation_size('enrollments')) AS table_size,
idx_demo->        pg_size_pretty(pg_relation_size('enrollments_student_id_idx')) AS
              index_size;
 table_size | index_size
------------+------------
 3336 kB    | 896 kB
(1 row)
```

The index takes about a quarter of the table's size, and it also slows down inserting, updating, and deleting rows: every change has to be reflected in the index too. That's why an index is created for a specific query, not “just in case”.

The index is part of the schema: adding it to schema.sql

The idx_demo database is throwaway, but the real one needs the foreign-key index. Add this to the end of schema.sql:

```
schema.sql — append to the end of the file
-- Index on the foreign key student_id (step 6): PostgreSQL doesn't create one.
CREATE INDEX enrollments_student_id_idx ON enrollments (student_id);
```

Go back to the main database and rebuild its contents. `DROP TABLE IF EXISTS` at the top of schema.sql drops the old tables, and seed.sql brings the data back:

```
psql
idx_demo=> \c course_catalog
You are now connected to database "course_catalog" as user "catalog_app".
course_catalog=> \i schema.sql
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
course_catalog=> \i seed.sql
INSERT 0 6
INSERT 0 3
INSERT 0 6
course_catalog=> \d enrollments
                                  Table "public.enrollments"
   Column    |           Type           | Collation | Nullable |           Default
-------------+--------------------------+-----------+----------+------------------------------
 id          | bigint                   |           | not null | generated always as identity
 course_id   | text                     |           | not null |
 student_id  | bigint                   |           | not null |
 enrolled_at | timestamp with time zone |           | not null | now()
Indexes:
    "enrollments_pkey" PRIMARY KEY, btree (id)
    "enrollments_course_id_student_id_key" UNIQUE CONSTRAINT, btree (course_id, student_id)
    "enrollments_student_id_idx" btree (student_id)
Foreign-key constraints:
    "enrollments_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE
        RESTRICT
    "enrollments_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE
        CASCADE
```

A third index has appeared in `\d enrollments` — `enrollments_student_id_idx`.

An index on a small table

Finally, in the main database, run a query on the primary key of the `courses` table:

```
psql
course_catalog=> ANALYZE courses;
ANALYZE
course_catalog=> EXPLAIN SELECT * FROM courses WHERE id = 'api-design';
                        QUERY PLAN
-----------------------------------------------------------
 Seq Scan on courses  (cost=0.00..1.07 rows=1 width=100)
   Filter: (id = 'api-design'::text)
(2 rows)
```

`courses.id` has a primary-key index, but PostgreSQL chose `Seq Scan`: a six-row table fits entirely on one data page, and reading it (cost 1.07) is cheaper than going through the index. The planner compares costs, and "there's an index" doesn't mean "the index will be used".

The result of step 6 — in the README

Add a section with the experiment's results to the README. Paste both plans (before and after) as code blocks, put the `EXPLAIN ANALYZE` numbers in the table, and answer three questions:

```
README.md — section template
## Index: EXPLAIN before and after

Query: `SELECT * FROM enrollments WHERE student_id = 777;`
Database idx_demo: 500 courses, 20,000 students, 50,000 enrollments.

| | Before the index | After the index |
|---|---|---|
| Access method | | |
| cost | | |
| Rows Removed by Filter | | |
| Execution Time | | |

<!-- TODO: paste both plans as code blocks -->

Answers to the questions:

1. TODO: why didn't PostgreSQL create an index on `enrollments.student_id`,
   although there are indexes on `email` and `course_id`?
2. TODO: why does a primary-key lookup on `courses` (six rows) use `Seq Scan`,
   while on `enrollments` (50 thousand rows) it goes through an index?
3. TODO: name two operations that an index on `student_id` will speed up.
```

A hint for question 3: recall what the database does when a student is deleted with a cascade (section 5) and which query returns one student's enrollments. Then make a commit:

```
terminal
git add schema.sql experiments/idx_data.sql README.md
git commit -m "Add index on enrollments.student_id"
```

7. A Transaction: Two Inserts as One Operation

Enrolling a new student in a course is two inserts: first a row in `students`, then a row in `enrollments`. If an error happens after the first insert, then without a transaction the database is left with a student who has no enrollment — “half an operation”. A transaction ties commands together into a whole: either all of them are applied (`COMMIT`) or none (`ROLLBACK`).

An operation that fails halfway

Start a transaction, insert a student, and then insert an enrollment in a nonexistent course: the second insert will violate the foreign key. The new student's number isn't known in advance (the database issues it), so we get it

with a query by email. Watch the prompt: `*` before `>` means "inside a transaction", `!` means "the transaction was aborted by an error". The verbose error mode from step 5 should be off (`\set VERBOSITY default`):

```
psql
course_catalog=> BEGIN;
BEGIN
course_catalog=*> INSERT INTO students (name, email)
course_catalog-*> VALUES ('Madina Ospanova', 'madina@example.com');
INSERT 0 1
course_catalog=*> INSERT INTO enrollments (course_id, student_id)
course_catalog-*> SELECT 'no-such-course', id FROM students
course_catalog-*> WHERE email = 'madina@example.com';
ERROR:  insert or update on table "enrollments" violates foreign key constraint
        "enrollments_course_id_fkey"
DETAIL:  Key (course_id)=(no-such-course) is not present in table "courses".
course_catalog=!> SELECT count(*) FROM students;
ERROR:  current transaction is aborted, commands ignored until end of transaction block
course_catalog=!> ROLLBACK;
ROLLBACK
course_catalog=> SELECT count(*) FROM students WHERE email = 'madina@example.com';
 count
-------
     0
(1 row)
```

- after an error the transaction is aborted: all following commands, even a harmless `SELECT`, are rejected with the message `current transaction is aborted, commands ignored until end of transaction block`. You get out of this state with `ROLLBACK`;
- after `ROLLBACK`, Madina isn't in the database (`count` = 0): the first insert was rolled back too, even though it was correct on its own. No student without an enrollment appeared.

The same operation, but successful: two windows

Now run both inserts correctly and see what another client sees. Open a second terminal window and start `psql -h localhost -U catalog_app -d course_catalog` in it — this is window B, the observer. Run the commands in the order shown below. Window A first:

```
psql — window A
course_catalog=> BEGIN;
BEGIN
course_catalog=*> INSERT INTO students (name, email)
course_catalog-*> VALUES ('Madina Ospanova', 'madina@example.com');
INSERT 0 1
course_catalog=*> INSERT INTO enrollments (course_id, student_id)
course_catalog-*> SELECT 'web-security', id FROM students
course_catalog-*> WHERE email = 'madina@example.com';
INSERT 0 1
```

The transaction in window A is open and both inserts have run, but they aren't committed. Look from window B:

```
psql — window B
course_catalog=> SELECT count(*) FROM students WHERE email = 'madina@example.com';
 count
-------
     0
(1 row)
```

Window B doesn't see Madina: changes are invisible to others until the transaction is committed. Commit it in window A and repeat the query in window B:

```
psql — window A
course_catalog=*> COMMIT;
COMMIT
```

```
psql — window B
course_catalog=> SELECT count(*) FROM students WHERE email = 'madina@example.com';
 count
-------
     1
(1 row)
```

After `COMMIT`, both rows appeared at once. Check the result in window A:

```
psql — window A
course_catalog=> SELECT s.name, c.title
course_catalog-> FROM enrollments e
course_catalog-> JOIN students s ON s.id = e.student_id
course_catalog-> JOIN courses c  ON c.id = e.course_id
course_catalog-> WHERE s.email = 'madina@example.com';
      name       |          title
-----------------+-------------------------
 Madina Ospanova | Web Security Essentials
(1 row)

course_catalog=> SELECT (SELECT count(*) FROM students) AS students,
course_catalog->        (SELECT count(*) FROM enrollments) AS enrollments;
 students | enrollments
----------+-------------
        4 |           7
(1 row)

course_catalog=> SHOW default_transaction_isolation;
 default_transaction_isolation
-------------------------------
 read committed
(1 row)
```

Window B sees only committed data: this isolation level is called `read committed` and is the default. Now the database has four students and seven enrollments, so the data differs from seed.sql; in section 8 you'll start from a clean slate.

> **An open transaction holds locks.** A prompt with * means the transaction isn't finished. While it's open, the `DROP TABLE` command from schema.sql in another window will wait for it to finish and “hang”. If psql doesn't respond, look for the window with * and run `ROLLBACK;` or `COMMIT;` there.

The result of step 7 — in the README

```
README.md — section template
## Transaction

Scenario: enroll a new student in a course — two inserts in one transaction.

- The second insert failed: TODO (what psql showed, what state the
  transaction was in, what happened to the first insert after ROLLBACK).
- Window B saw TODO before COMMIT and TODO after COMMIT.
- TODO: what would have happened without a transaction if the second
  insert had failed?
```

> **One API request, one short transaction.** In week 7, a SQLAlchemy session will open a transaction, and the route handler will end it with `commit()` or roll it back on an exception. Transactions are kept short: while one is open, the rows it changed can't be changed by anyone else.

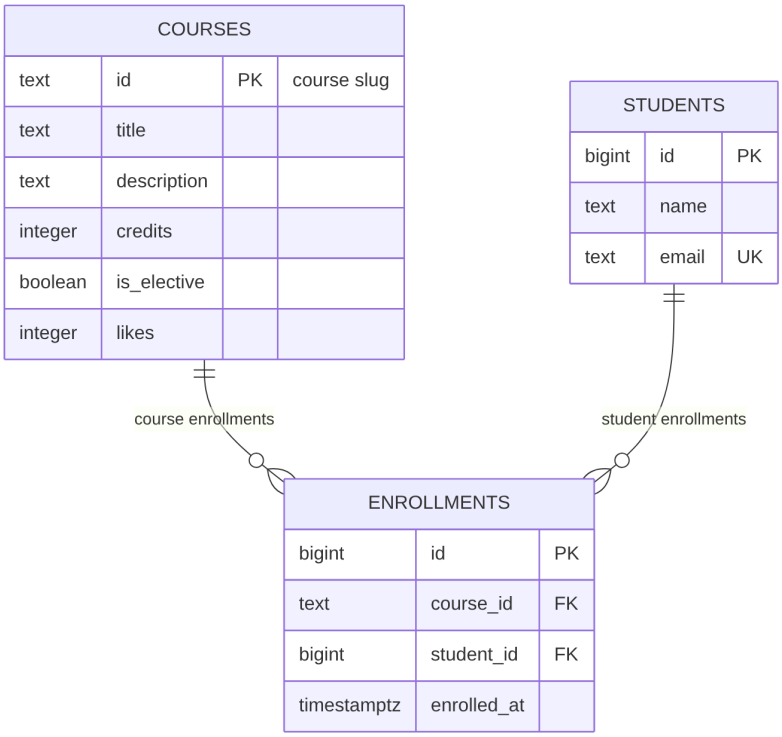
```
terminal
git add README.md
git commit -m "Add transaction notes"
```

8. The Repository: README, ER Diagram, and Checklist

What's left is to assemble everything into a repository that a reviewer can run from scratch, and to review the schema against the checklist.

The ER diagram in the README

An ER diagram shows the tables, their columns, and the relationships. It's convenient to write it as code: GitHub turns a code block marked `mermaid` in the README into a picture. This is what the diagram of your schema looks like, and in the README you paste its text description:



The schema's ER diagram: courses and students are linked to enrollments by a one-to-many relationship

Notation	What it means
`COURSES ||--o{ ENROLLMENTS : "course enrollments"`	a one-to-many relationship: a course has many enrollments, an enrollment has one course; the quoted text is the relationship's label
`||` and `o{`	`||` — exactly one; `o{` — zero or many
`text id PK "course slug"`	a column row: type, name, key (`PK`, `FK`, `UK`), and an optional quoted comment

README.md — skeleton of the “Schema” section

```
## Schema

```mermaid
erDiagram
    COURSES ||--o{ ENROLLMENTS : "course enrollments"
    %% TODO 1: the STUDENTS — ENROLLMENTS relationship
    %%         (a student has many enrollments)
    COURSES {
        text id PK "course slug"
        text title
        text description
        integer credits
        boolean is_elective
        integer likes
    }
    %% TODO 2: the STUDENTS block (id, name, email; keys PK and UK)
    %% TODO 3: the ENROLLMENTS block (id, course_id, student_id, enrolled_at;
    %%         keys PK, FK, FK)
```

TODO 4: 2–3 sentences: the tables, how they're linked, where “many-to-many” is.
```

> **The diagram didn't render?** Open the README on GitHub after `git push`. If you see text or an error instead of a picture, check that the block starts with exactly three backticks and the word `mermaid`, that there are no typos inside, and that each column line (`type name`) has two words.

The rest of the README

A reviewer should be able to run the project from scratch using the README. It consists of seven sections; you've already written three of them as you went:

README section	What's in it
How to run	commands for an empty server: the role and database, then the three files with `-v ON_ERROR_STOP=1` (skeleton below)
Schema	the Mermaid ER diagram and 2–3 sentences (above)
Design decisions	answers to three questions (skeleton below)
Constraint violation log	the table and the Pydantic answer (step 5)
Index: EXPLAIN before and after	the plans, the table, and three answers (step 6)
Transaction	what the two windows showed (step 7)
Self-review against the checklist	a table of seven rows (below)

README.md — the start of the file and the "How to run" section

````
# course-catalog-db

PostgreSQL schema for the course catalog (Lab 6, "Advanced Web Technologies").

## How to run

PostgreSQL 16 or newer is required. Once, as the `postgres` superuser:

```sql
CREATE ROLE catalog_app LOGIN PASSWORD 'change-me';
CREATE DATABASE course_catalog OWNER catalog_app;
```

Then, from the project folder (it stops at the first error):

```bash
psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f schema.sql
psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f seed.sql
psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f queries.sql
```

The password `change-me` is for teaching; real projects keep passwords out of git.
````

README.md — the "Design decisions" section

```
## Design decisions

- TODO: why `courses.id` is text (a slug) while `students.id` is an automatic
  number; the pros and cons of each approach.
- TODO: why `RESTRICT` on `enrollments.course_id` but `CASCADE` on
  `enrollments.student_id`.
- TODO: why `timestamptz` and not `timestamp`, and why `text` and not
  `varchar(n)`.
```

Checklist: review your schema

In class, you'll review someone else's schema against the same list; before submitting, review your own against it. For each row, write “yes” or “no” and one comment in the README — for example, which columns you made NOT NULL and why:

What to check	The question to ask
Keys	Does every table have a primary key? Is there a foreign key for every relationship?
Types	Does each type fit its value: money — numeric, moments in time — timestamptz, identifiers — bigint?
Required values	Which columns must be NOT NULL? Is NULL allowed anywhere a value is required?
Constraints	Are the domain rules written as UNIQUE and CHECK? Are they consistent with Pydantic?
Deletion	What happens to child rows when a parent is deleted? Were RESTRICT or CASCADE chosen deliberately?
Normalization	Is any fact stored in several places? Are there lists of values in a single cell?
Indexes and names	Are there indexes on foreign keys and on frequent queries, and are there any unneeded ones? Are names consistent (snake_case, courses, course_id)?

README.md — the “Self-review against the checklist” section

```
## Self-review against the checklist

| What to check | Result |
|---------------|--------|
| Keys | |
| Types | |
| Required values | |
| Constraints | |
| Deletion | |
| Normalization | |
| Indexes and names | |
```

A clean-slate check

A repository works only if the project runs from the README without manual fixes of your own. Run the three files as written there: the `-v ON_ERROR_STOP=1` flag stops execution at the first error, so you see right away where it is. You already have tables, so schema.sql drops them first (on a completely empty database, three familiar NOTICE lines would appear before the first `DROP TABLE`). The first two files should print:

terminal — from the project folder

```
$ psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f schema.sql
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
$ psql -h localhost -U catalog_app -d course_catalog -v ON_ERROR_STOP=1 -f seed.sql
INSERT 0 6
INSERT 0 3
INSERT 0 6
```

The third (`queries.sql`) prints the results of all the queries; make sure none of them is an ERROR message. If any file stopped, fix it and repeat all three commands from the start. After that, the data is the same as after seed.sql again (6, 3, and 6 rows).

Publishing to GitHub

Create an empty `course-catalog-db` repository on GitHub (no README and no .gitignore), and then push the project to it. Check the commit log: there should be at least six commits, and the messages should explain what changed:

```
terminal
git add .
git commit -m "Finish README and ER diagram"
git branch -M main
git remote add origin https://github.com/<your-username>/course-catalog-db.git
git push -u origin main
git log --oneline
```

Open the repository on GitHub: the README should show the diagram, and the file list should include schema.sql, seed.sql, queries.sql, the experiments folder, and README.md.

Acceptance Checklist

Before you submit, run through this list — it's exactly what will be checked:

- ☐ `schema.sql` runs without errors with `psql … -v ON_ERROR_STOP=1 -f schema.sql`, including a repeat run on top of existing tables; it has three tables (`courses`, `students`, `enrollments`) with all the columns from the rules table in section 2;
- ☐ the constraints are in place: `NOT NULL`, `DEFAULT`, `CHECK` (`credits > 0`, `likes >= 0`, a name of at least two characters), `UNIQUE` (email and the pair `course_id`, `student_id`), primary keys on all the tables, and two foreign keys — `RESTRICT` on `course_id`, `CASCADE` on `student_id`; they are visible in `\d` of each table;
- ☐ after `schema.sql` and `seed.sql`, the tables hold 6 courses, 3 students, and 6 enrollments;
- ☐ `queries.sql` runs in full without errors and gives the same results on a repeat run (except the enrollment number in query 4); queries 3 and 4 are wrapped in `BEGIN … ROLLBACK`; the statistics query uses `count(e.id)`;
- ☐ the README has a violation log: ten experiments and a cascading delete; for each one, the SQLSTATE, the constraint name, and the API response are correct; the Pydantic question is answered;
- ☐ `schema.sql` contains `CREATE INDEX enrollments_student_id_idx`, and `experiments/idx_data.sql` creates the data for the experiment; the README has both `EXPLAIN` plans (before and after), a table with `Execution Time`, and three answers;
- ☐ the README describes the transaction experiment: the rollback of both inserts after an error and the successful variant with two windows (what the second client saw before and after `COMMIT`);
- ☐ the README has a Mermaid ER diagram that renders on GitHub, a “Design decisions” section with three answers, and a completed checklist;

- ☐ the project runs from scratch using the README: it has the commands for the role, the database, and the three files;
- ☐ the `course-catalog-db` repository is on GitHub and has at least six meaningful commits.

Grading Rubric

Criterion	Points
`schema.sql` and `seed.sql`: three tables, types, keys, all the rules from section 2, foreign keys with `RESTRICT` and `CASCADE`; the files run repeatedly, and the tables hold 6, 3, and 6 rows	2
`queries.sql`: the nine queries give the results from section 4; the modifying queries are wrapped in `BEGIN … ROLLBACK`; the statistics query uses `count(e.id)`	2
Violation log: ten experiments and the cascade, correct SQLSTATEs, constraint names, and API responses; the answer about Pydantic	2
Index: `EXPLAIN` plans before and after, the time, the index added to `schema.sql`, three answers in the README	1
Transaction: the rollback of both inserts and the successful two-window scenario are described and explained	1
Repository: README (how to run, ER diagram, decisions, checklist), at least six meaningful commits	2

Total: 10 points. Adjust the scale to your own grading system as needed.

Common Mistakes

> **permission denied for schema public when creating tables:** the database wasn't created with `OWNER catalog_app` (for example, with a plain `CREATE DATABASE course_catalog;`). Since PostgreSQL 15, an ordinary role can't create tables in the `public` schema of someone else's database. The message looks like this:

```
psql — shown with a short command
course_catalog=> CREATE TABLE courses (id text PRIMARY KEY);
ERROR:  permission denied for schema public
LINE 1: CREATE TABLE courses (id text PRIMARY KEY);
                     ^
```

Fix it as the `postgres` superuser and run schema.sql again:

```
psql — superuser
ALTER DATABASE course_catalog OWNER TO catalog_app;
```

psql won't connect. The message begins with `psql: error: connection to server at "localhost" (127.0.0.1), port 5432 failed:`, and the cause is in what follows:

What follows `failed:`	Cause and what to do
`Connection refused` and the line `Is the server running on that host and accepting TCP/IP connections?`	the server isn't running or is listening on a different port: start the PostgreSQL service (in Docker — `docker start pg`) and check the port
`FATAL: role "catalog_app" does not exist`	the role wasn't created (step 1), or you connected to a different server than the one where you created it
`FATAL: database "course_catalog" does not exist`	the database wasn't created, or its name has a typo: check `\l` as the superuser and repeat `CREATE DATABASE`

syntax error at or near … in schema.sql: almost always an extra or a missing comma. The most common mistake is a comma after the last element, before `)`:

```
psql
course_catalog=> \i schema.sql
psql:schema.sql:4: NOTICE:  table "enrollments" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "students" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "courses" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
psql:schema.sql:27: ERROR:  syntax error at or near ")"
LINE 7: );
        ^
```

`schema.sql:27` is the line of the file where the command broke off; `LINE 7` is the line inside the command itself, and the `^` marks the spot (your numbers will be different). Notice that the first two tables have already been created and the third hasn't: the file isn't rolled back as a whole. Fix the line and run the file again — `DROP TABLE IF EXISTS` at its start will first drop whatever managed to get created. If a comma is missing, the message points to the word that comes right after the place where it was left out.

DROP TABLE courses complains about dependencies: you can't drop a table that another table's foreign key refers to. Don't drop the tables one at a time, and don't add `CASCADE` at random (it would remove the foreign keys of the dependent tables): schema.sql drops all three tables with a single command, in the right order.

```
psql
course_catalog=> DROP TABLE courses;
ERROR:  cannot drop table courses because other objects depend on it
DETAIL:  constraint enrollments_course_id_fkey on table enrollments depends on table courses
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

Running seed.sql again floods you with errors: the inserts violate uniqueness because the rows are already there. Not a single row was added, and the data is unchanged (6, 3, and 6): `UNIQUE` and the primary keys protected it (the line numbers in your messages will be your own). To get back to a clean state, run `\i schema.sql` and then `\i seed.sql`.

```
psql
course_catalog=> \i seed.sql
psql:seed.sql:15: ERROR:  duplicate key value violates unique constraint "courses_pkey"
DETAIL:  Key (id)=(modern-frontend) already exists.
psql:seed.sql:20: ERROR:  duplicate key value violates unique constraint "students_email_key"
DETAIL:  Key (email)=(aigerim@example.com) already exists.
psql:seed.sql:25: ERROR:  duplicate key value violates unique constraint
        "enrollments_course_id_student_id_key"
DETAIL:  Key (course_id, student_id)=(modern-frontend, 1) already exists.
```

`\i shema.sql`: No such file or directory: psql wasn't started from the project folder, or the file name has a typo. Quit (`\q`), use `cd` to go to the project folder, and start psql again. For files in subfolders, the path is written relative to that same folder: `\i experiments/idx_data.sql`.

```
psql
course_catalog=> \i shema.sql
shema.sql: No such file or directory
```

Cyrillic letters turned into garbage: the psql client wasn't working in UTF-8 (most often on Windows), and text with Cyrillic or other non-English letters was written to the database garbled — without a single error. Suppose a student's name is inserted and then read back like this. The garbage can look different; here is one variant:

```
psql
course_catalog=> INSERT INTO students (name, email)
course_catalog-> VALUES ('Айгерим Садыкова', 'aigerim.kz@example.com');
INSERT 0 1
course_catalog=> SELECT name FROM students WHERE email = 'aigerim.kz@example.com';
              name
--------------------------------
 РђРѕРіРµСЂРёРј РЎР°РґС‹РєРѕРІР°
(1 row)
```

The fix: run `\encoding UTF8` (check with `SHOW client_encoding;` — the answer is `UTF8`) and write the text again: delete the garbled row and insert it anew, because what is already stored doesn't repair itself. If the text came from your own file, run `\i schema.sql` and `\i seed.sql` again: the tables will be recreated with the correct text. On Windows, set `PGCLIENTENCODING=UTF8` in advance, as in section 1.

psql "hangs" on schema.sql or on another command: another window has an open transaction (a prompt with `*`) that has already touched these tables, and `DROP TABLE` waits for it to finish. Find the window with `*` and run `ROLLBACK;` there, and then check that the prompt in every window has no asterisk.

Bonus Tasks (optional)

If the core part is done ahead of time, pick any of these — there's no limit on how many. Do them on clean data: after the clean-slate check (section 8), that's what you have. Describe the results briefly in the README, in a “Bonus” section.

Task 1. Normalization: three anomalies of a flat table

In section 8 of the lecture, we looked at a flat `registrations` table, where every row repeats the student's name and email and the course's title and credits. Create the file experiments/norm_demo.sql and load it into the throwaway database `norm_demo`:

```
experiments/norm_demo.sql
-- norm_demo.sql: a flat registrations table for normalization experiments
DROP TABLE IF EXISTS registrations;

CREATE TABLE registrations (
    id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_name  text,
    student_email text,
    course_id     text,
    course_title  text,
    credits       integer
);

INSERT INTO registrations
  (student_name, student_email, course_id, course_title, credits) VALUES
  ('Aigerim Sadykova', 'aigerim@example.com',
   'modern-frontend', 'Modern Frontend: React & Next.js', 5),
  ('Aigerim Sadykova', 'aigerim@example.com',
   'ai-integration', 'AI/LLM Integration', 5),
  ('Daniyar Bekov', 'daniyar@example.com',
   'modern-frontend', 'Modern Frontend: React & Next.js', 5),
  ('Daniyar Bekov', 'daniyar@example.com',
   'backend-fastapi', 'Backend Foundations: FastAPI', 5),
  ('Daniyar Bekov', 'daniyar@example.com',
   'ai-integration', 'AI/LLM Integration', 5),
  ('Asel Omarova', 'asel@example.com',
   'ai-integration', 'AI/LLM Integration', 5);
```

```
psql
course_catalog=> \c norm_demo
You are now connected to database "norm_demo" as user "catalog_app".
norm_demo=> \i experiments/norm_demo.sql
psql:experiments/norm_demo.sql:2: NOTICE:  table "registrations" does not exist, skipping
DROP TABLE
CREATE TABLE
INSERT 0 6
norm_demo=> SELECT id, student_name, course_id, course_title FROM registrations ORDER BY id;
 id |   student_name   |    course_id    |           course_title
----+------------------+-----------------+---------------------------------
  1 | Aigerim Sadykova | modern-frontend | Modern Frontend: React & Next.js
  2 | Aigerim Sadykova | ai-integration  | AI/LLM Integration
  3 | Daniyar Bekov    | modern-frontend | Modern Frontend: React & Next.js
  4 | Daniyar Bekov    | backend-fastapi | Backend Foundations: FastAPI
  5 | Daniyar Bekov    | ai-integration  | AI/LLM Integration
  6 | Asel Omarova     | ai-integration  | AI/LLM Integration
(6 rows)
```

Run three experiments. Each is done inside `BEGIN … ROLLBACK`, so the table stays as it was. The first is the update anomaly: rename the course in just one row.

```
psql
norm_demo=> BEGIN;
BEGIN
norm_demo=*> UPDATE registrations
norm_demo-*> SET course_title = 'Modern Frontend: React 19 & Next.js 16'
norm_demo-*> WHERE student_name = 'Daniyar Bekov' AND course_id = 'modern-frontend';
UPDATE 1
norm_demo=*> SELECT DISTINCT course_id, course_title FROM registrations
norm_demo-*> WHERE course_id = 'modern-frontend';
    course_id    |              course_title
-----------------+----------------------------------------
 modern-frontend | Modern Frontend: React & Next.js
 modern-frontend | Modern Frontend: React 19 & Next.js 16
(2 rows)

norm_demo=*> ROLLBACK;
ROLLBACK
```

The `modern-frontend` course now has two titles, and the database doesn't know which one is right: to rename the course properly, you would have to update all of its rows. The second experiment is the delete anomaly: delete the only enrollment in the FastAPI course.

```
psql
norm_demo=> BEGIN;
BEGIN
norm_demo=*> DELETE FROM registrations
norm_demo-*> WHERE student_name = 'Daniyar Bekov' AND course_id = 'backend-fastapi';
DELETE 1
norm_demo=*> SELECT * FROM registrations WHERE course_id = 'backend-fastapi';
 id | student_name | student_email | course_id | course_title | credits
----+--------------+---------------+-----------+--------------+---------
(0 rows)

norm_demo=*> ROLLBACK;
ROLLBACK
```

Along with the enrollment, the `backend-fastapi` course itself disappeared: the course's details lived only inside the enrollments. The third experiment is the insert anomaly: add a course that nobody has enrolled in yet.

```
psql
norm_demo=> BEGIN;
BEGIN
norm_demo=*> INSERT INTO registrations (course_id, course_title, credits)
norm_demo-*> VALUES ('web-security', 'Web Security Essentials', 4);
INSERT 0 1
norm_demo=*> SELECT student_name, course_id FROM registrations WHERE course_id =
                 'web-security';
 student_name |  course_id
--------------+--------------
              | web-security
(1 row)

norm_demo=*> ROLLBACK;
ROLLBACK
```

We had to insert a row with an empty student: without an enrollment, there's nowhere to keep a course. Now move the data into normalized tables. First create them in this same database with `\i schema.sql`, then fill the tables with three `INSERT … SELECT` queries:

```
psql
norm_demo=> \i schema.sql
psql:schema.sql:4: NOTICE:  table "enrollments" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "students" does not exist, skipping
psql:schema.sql:4: NOTICE:  table "courses" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
norm_demo=> INSERT INTO students (name, email)
norm_demo-> SELECT student_name, student_email FROM registrations
norm_demo-> GROUP BY student_name, student_email
norm_demo-> ORDER BY min(id);
INSERT 0 3
norm_demo=> INSERT INTO courses (id, title, description, credits)
norm_demo-> SELECT DISTINCT course_id, course_title, '', credits FROM registrations;
INSERT 0 3
norm_demo=> INSERT INTO enrollments (course_id, student_id)
norm_demo-> SELECT r.course_id, s.id
norm_demo-> FROM registrations r
norm_demo-> JOIN students s ON s.email = r.student_email
norm_demo-> ORDER BY r.id;
INSERT 0 6
norm_demo=> SELECT s.name, c.title
norm_demo-> FROM enrollments e
norm_demo-> JOIN students s ON s.id = e.student_id
norm_demo-> JOIN courses c  ON c.id = e.course_id
norm_demo-> ORDER BY e.id;
       name       |              title
------------------+---------------------------------
 Aigerim Sadykova | Modern Frontend: React & Next.js
 Aigerim Sadykova | AI/LLM Integration
 Daniyar Bekov    | Modern Frontend: React & Next.js
 Daniyar Bekov    | Backend Foundations: FastAPI
 Daniyar Bekov    | AI/LLM Integration
 Asel Omarova     | AI/LLM Integration
(6 rows)

norm_demo=> SELECT (SELECT count(*) FROM courses) AS courses,
norm_demo->        (SELECT count(*) FROM students) AS students,
norm_demo->        (SELECT count(*) FROM enrollments) AS enrollments;
 courses | students | enrollments
---------+----------+-------------
       3 |        3 |           6
(1 row)
```

- students: `GROUP BY` leaves one row per student, and `ORDER BY min(id)` issues the numbers in the order of their first enrollments;
- courses: `SELECT DISTINCT` removes the repeats; the flat table had no descriptions, so an empty string `''` is inserted (it doesn't violate `NOT NULL`);
- enrollments: `JOIN students` on email finds the student's new number.

There are three courses, not six: the three courses without enrollments (`api-design`, `databases-postgresql`, `web-security`) had nowhere to live in the flat table, while the normalized schema stores them on their own —

that is the insert anomaly in action. In the README, describe which three anomalies you saw and how splitting the data into tables removes each one.

Task 2. A lost update and a row lock

In section 4, a like was incremented right inside the `UPDATE` command. Find out by experiment what's wrong with the "read — compute in the program — write back" scheme. Return to the main database (`\c course_catalog`) and open a second psql window: both windows are connected to course_catalog. Each window "reads" the number of likes, as code with `course.likes + 1` would:

```
psql — window A
norm_demo=> \c course_catalog
You are now connected to database "course_catalog" as user "catalog_app".
course_catalog=> SELECT likes FROM courses WHERE id = 'api-design';
 likes
-------
    11
(1 row)
```

```
psql — window B
course_catalog=> SELECT likes FROM courses WHERE id = 'api-design';
 likes
-------
    11
(1 row)
```

Both got 11. Now each window writes its own value, 12:

```
psql — window A
course_catalog=> UPDATE courses SET likes = 12 WHERE id = 'api-design';
UPDATE 1
```

```
psql — window B
course_catalog=> UPDATE courses SET likes = 12 WHERE id = 'api-design';
UPDATE 1
```

```
psql — window A
course_catalog=> SELECT likes FROM courses WHERE id = 'api-design';
 likes
-------
    12
(1 row)
```

Two likes, but the number went up by one: both clients read the same value and wrote the same value, and one like was lost. Put 11 back (`UPDATE courses SET likes = 11 WHERE id = 'api-design';`) and repeat the experiment, incrementing the value inside the command:

```
psql — window A
course_catalog=> UPDATE courses SET likes = likes + 1 WHERE id = 'api-design' RETURNING likes;
 likes
-------
    12
(1 row)

UPDATE 1
```

```
psql — window B
course_catalog=> UPDATE courses SET likes = likes + 1 WHERE id = 'api-design' RETURNING likes;
 likes
-------
    13
(1 row)

UPDATE 1
```

```
psql — window A
course_catalog=> SELECT likes FROM courses WHERE id = 'api-design';
 likes
-------
    13
(1 row)
```

Now both likes are there. Why this works is visible in the row lock. Put 11 back again, then in window A open a transaction and increment the counter, but don't commit it:

```
psql — window A
course_catalog=> BEGIN;
BEGIN
course_catalog=*> UPDATE courses SET likes = likes + 1 WHERE id = 'api-design' RETURNING
                      likes;
 likes
-------
    12
(1 row)

UPDATE 1
```

In window B, run the same command. There's no response: the row is held by A's transaction, and psql waits. Commit the transaction in window A:

```
psql — window A
course_catalog=*> COMMIT;
COMMIT
```

Right after that, a response appears in window B: the command ran and returned 13, not 12. Having waited for the lock to be released, B re-read the fresh value (12) and added one to it:

```
psql — window B (the response came after COMMIT in window A)
course_catalog=> UPDATE courses SET likes = likes + 1 WHERE id = 'api-design' RETURNING likes;
 likes
-------
    13
(1 row)

UPDATE 1
```

Reset the likes to 11. In the README, explain how `UPDATE … SET likes = likes + 1` differs from reading the number into the program and writing back a new one.

Task 3. Parameters instead of concatenation: SQL injection

Imagine that a course identifier comes from a user and the program substitutes it into the query text. The user enters `x' OR '1'='1`, and this is the query that results:

```
psql
course_catalog=> SELECT id, title FROM courses WHERE id = 'x' OR '1'='1';
          id          |              title
----------------------+---------------------------------
 modern-frontend      | Modern Frontend: React & Next.js
 backend-fastapi      | Backend Foundations: FastAPI
 databases-postgresql | Relational Databases: PostgreSQL
 web-security         | Web Security Essentials
 ai-integration       | AI/LLM Integration
 api-design           | API Design: REST vs GraphQL
(6 rows)
```

The condition became true for every row, and the database returned all the courses. If a password were checked this way, anyone could log in. A parameter, on the other hand, is passed separately from the query text, and the database always reads it as a value. Check with `PREPARE` and `EXECUTE`:

```
psql
course_catalog=> PREPARE by_id (text) AS
course_catalog->   SELECT id, title FROM courses WHERE id = $1;
PREPARE
course_catalog=> EXECUTE by_id('x'' OR ''1''=''1');
 id | title
----+-------
(0 rows)

course_catalog=> EXECUTE by_id('api-design');
     id     |            title
------------+----------------------------
 api-design | API Design: REST vs GraphQL
(1 row)
```

The whole string `x' OR '1'='1` is just a value to compare with `id`; there's no such course, so the answer is empty. The drivers you'll use in week 7 pass values exactly this way. Release the prepared statement with `DEALLOCATE by_id;`. In the README, answer: why is the second variant safe?

Task 4. Two more queries: courses without enrollments, and HAVING

Add two queries to queries.sql and check them against the results.

- query 7: the `id`s of the courses nobody is enrolled in, in alphabetical order. Hint: `LEFT JOIN` to `enrollments` and the condition `WHERE e.id IS NULL` — it keeps only the courses for which no matching row was found;
- query 8: a student's name and the number of their courses (`AS courses`), for students enrolled in more than one course; those with more courses first. Hint: `GROUP BY s.id` and `HAVING count(*) > 1` — `HAVING` selects groups, while `WHERE` selects rows before grouping.

```
expected result of queries 7 and 8
          id
----------------------
 api-design
 databases-postgresql
 web-security
(3 rows)

       name       | courses
------------------+---------
 Daniyar Bekov    |       3
 Aigerim Sadykova |       2
(2 rows)
```

Task 5. A new rule for a live table: ALTER TABLE

The customer asks: a student's email address must contain `@`. There's no need to recreate the table and lose data: the rule is added with `ALTER TABLE … ADD CONSTRAINT`. The database first checks the existing rows and applies the rule only if all of them satisfy it:

```
psql
course_catalog=> ALTER TABLE students
course_catalog->   ADD CONSTRAINT students_email_format CHECK (position('@' IN email) > 1);
ALTER TABLE
course_catalog=> INSERT INTO students (name, email) VALUES ('Test User', 'no-at-sign');
ERROR:  new row for relation "students" violates check constraint "students_email_format"
DETAIL:  Failing row contains (4, Test User, no-at-sign).
```

Then add the same rule to `CREATE TABLE students` in schema.sql, so that the schema in the repository matches the database. In real projects, such changes are made as migrations (Alembic, week 7); it's better to give the constraint a name explicitly, as here, so the error is easy to recognize by that name.

How to Submit

One repository is submitted — `course-catalog-db`. It's new: create it empty on GitHub and push the project with the commands from section 8. This lab doesn't change the course-catalog and course-catalog-api projects.

Commit as you go, not in one chunk at the end. Naming guideline: Add schema.sql with constraints → Add seed data → Add queries for API routes → Add constraint violation log → Add index on enrollments.student_id → Add transaction notes → Finish README and ER diagram. The required minimum is 6 commits.

The password `change-me` is for teaching, so it's acceptable in the README; real passwords don't go into a repository (in week 7, we'll move them into environment variables).

Send the link to the repository however your instructor specifies (LMS / course chat).

Deadline: before Lecture 7 begins, unless your instructor states otherwise.